

Module 2:- Exception Handling.

30/7/26

Exception : The errors generated at runtime is called an exception. (or) Runtime errors are called as exception.

If these errors occurs the program terminates abruptly. To continue the normal flow of the program so that all the statements get executed, is called exception handling

Examples:- 1) Dividing a number by zero.

- 2) Accessing array element which is out of boundary
- 3) Accessing an array element with a negative index number
- 4) Connecting to a system with wrong ip address.
- 5) Using a wrong driver to connect the database.
- 6) Accessing a file which doesnot exist.

Q. How to handle exceptions in java?

By using 5 keywords - try, catch, finally, throw, throws.

Program

```
public class Excep
```

```
{  
    public static void main (String [] args)
```

```
{  
    int a=10, b=0;
```

```
    S.o.p(a/b);
```

```
    S.o.p("I am printed"); } }
```

Using try catch Block :-

Syntax: try
 {

 }

 catch ()

 {

 }

Note:- the try block should contain all trouble statements which may raise an exception.

The catch block to be followed after the try block will handle the exception raised by the try block with a suitable handler.

Q How many ways an exception message can be thrown?

Ans Three ways -

1) Using an object of exception class

It prints class name along with message.

2) Using getMessage():

3) Using printStackTrace():

which prints line number where exception is raised, exception class name, message.

Q. Write a program to demonstrate
ArrayIndexOutOfBoundsException

Q. Can a try block can have multiple catch blocks.

Finally

11/8/26

- A finally block are guaranteed of execution where exception is raised or not by try block, if raised successfully handled or not handled by the catch block.
- These statements such as closing of the streams, closing of files, closing the database connections and free up system resources.

Note

A try block must be followed either by catch block or by finally block.

class finally

```
{  
    public static void main (String[] args)
```

```
{  
    try {
```

```
        System.out.println (10/0);
```

```
    }
```

```
    catch (NPE e)
```

```
    {  
        System.out.println (e);
```

```
    }
```

```
    finally
```

```
    {  
        System.out.println ("I am printed");
```

```
    }
```


System.out.println ("I ~~target~~ executed");

↳

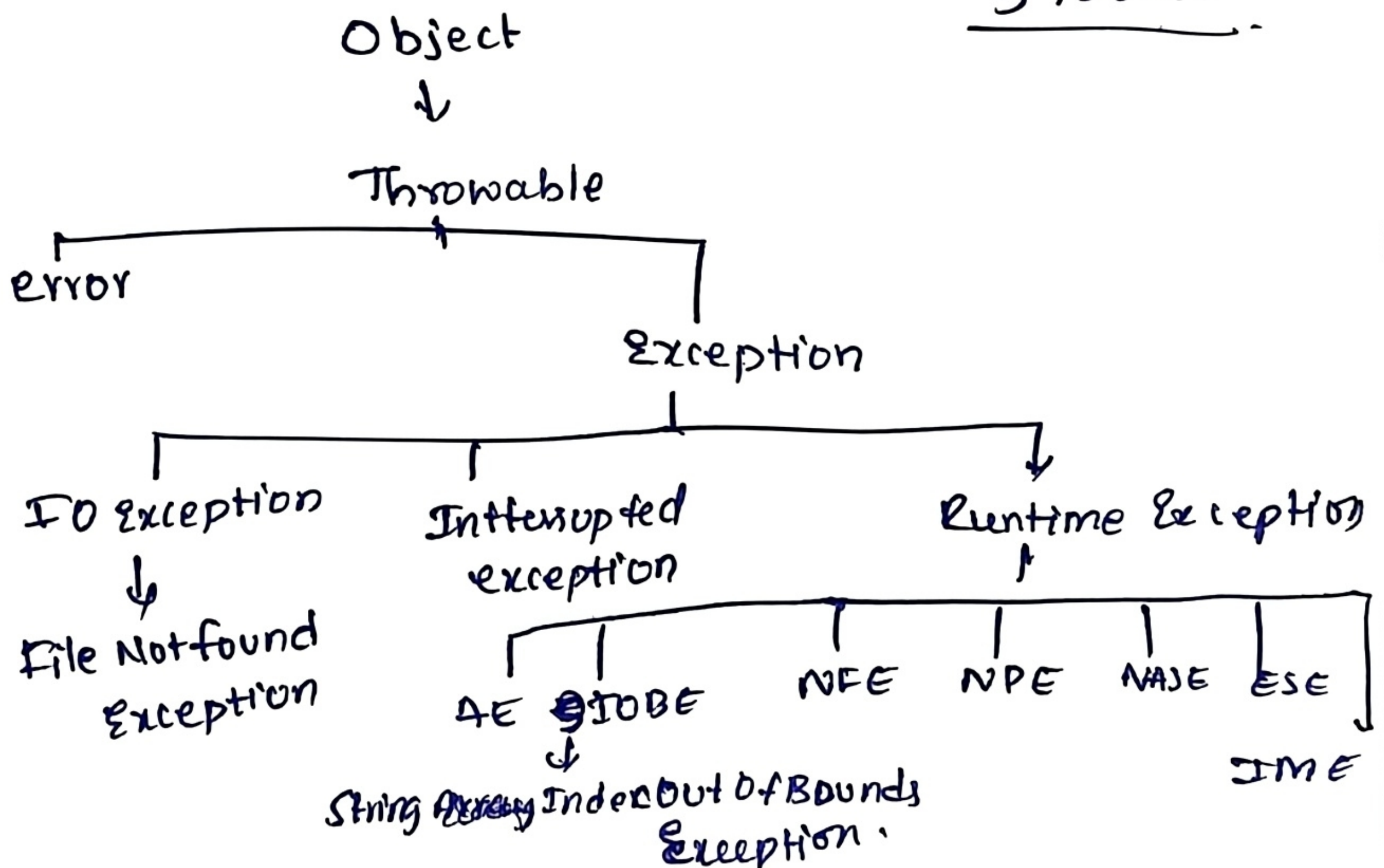
- A finally keyword is used with variables, methods & classes.
- Refer to inheritance notes.
- finalizer is a method used for freeing up resources before object is garbage collected.

Adv

Nested by catch

- IF a try catch block is declared within try block / in catch block.
- Using nested try-catch block we can throw more than one exceptions sequentially.

3108126



Types of Exceptions:-

1) Checked Exceptions

2) Unchecked Exceptions.

1) Checked Exceptions: the sub classes of Exception class except runtime exception classes are called checked exceptions.

If checked exception exists the program does not compile.

2) Unchecked Exceptions: The sub classes of runtime exceptions are called unchecked exceptions. If unchecked exception exist the program compiles but do not execute

~~class~~

Throws keyword:-

It is a keyword which is used for to display multiple exceptions of a method.

It is an alternative for try catch block and works only for checked exceptions.

Syntax:-

AccessSpecifier returnType methodName throws

Exception1, Exception2 ... Exceptionn

{ block of code }

}

* throw keyword :-

throw is a keyword which is used to throw an exception explicitly.

Syntax:

- throw instance of an exception.

⇒ NOTE :-

throw is a keyword which is used for throwing an user-defined exception or custom-defined exception.

Q. Write a java program to create InvalidMarksException

```
import java.util.*;
```

```
class InvalidMarksException extends Exception
```

```
{  
    public InvalidMarksException (String msg)
```

```
{  
        super(msg);
```

```
}
```

```
}
```

```
public class Marks
```

```
{  
    public static void main (String[] args) throws
```

```
InvalidMarksException {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println ("Enter your marks ");
```



```
int x = sc.nextInt();
```

```
if (x < 0 || x > 100)
```

```
{  
    throw new InvalidMarkException ("Marks cannot  
        be -ve value and above total marks");  
}
```

```
  
else
```

```
{  
    System.out.println ("print your marks " + x);  
}
```

```
  
System.out.println ("example for user defined  
                        exception");  
}
```

```
  
}
```

Q. Write a java program to create Negative Number
Exception ~~when~~

Strings

04/08/26

String is a group of characters which is derived from a class string class

The string class is defined in java.lang package

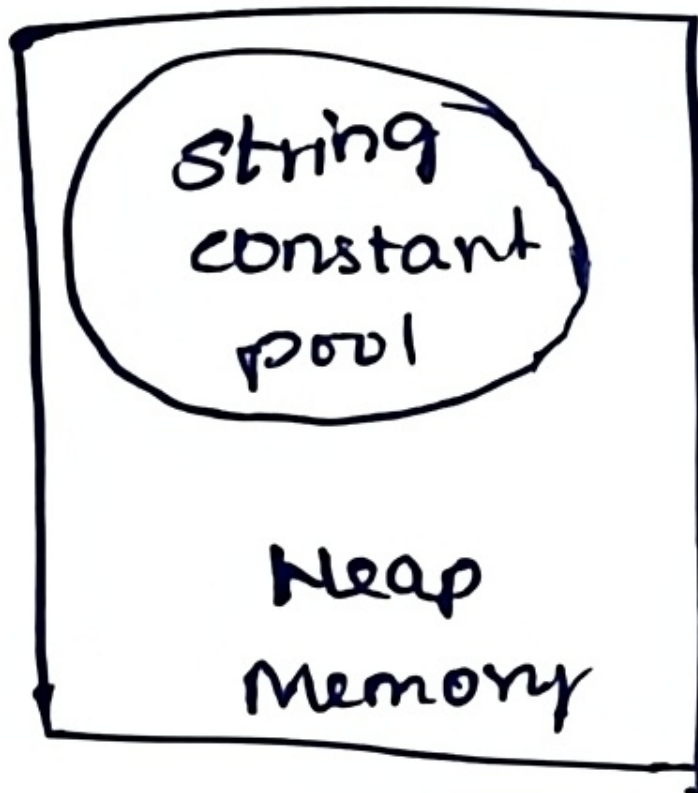
In Java, strings can be handled using three classes

- 1) String class
- 2) StringBuffer class
- 3) StringBuilder class (1.5 version)

How many ways a string can be created?

Two ways -

- 1) Using StringLiteral eg: String s1 = "Vardhaman";
- 2) Using new keyword eg: String s2 = new String("H sec");



When we create using string literal, it is stored in string constant pool. where as using new keyword, it stores in heap memory.

```
char CC[] = { 'J', 'A', 'V', 'A' };
```


NOTE:-

strings are immutable in nature

Methods of String class:-

- 1) ~~char~~ at takes a input as index and returns the character at the specified index.
- 2) The index of method ~~takes~~ is a overloading method which takes two types of inputs -
1) character, 2) string. and it always returns index of the first character. for both inputs and. it always returns first occurrence of character
- 3) trim() is used to remove whitespace characters.
- 4) substring() is a overloaded method which extracts part of a string it takes start index as well as Endindex (it reads one past $n-1$)

5/8/2026

Q. Write a java program to create file-format Exception (FRE) which should accept only Pdf files.

```
import java.util.*;

class FileFormatException extends Exception {
    public FileFormatException (String msg) {
        super(msg);
    }
}

public class Pdf {
    public static void main (String [] args) throws FileFormatException {
        Scanner sc = new Scanner (System.in);
        String s = sc.nextLine();
        boolean b = s.endsWith ("pdf");
        if (b == true) {
            System.out.println ("Should accept pdf files");
        }
        else {
            System.out.println ("Shouldn't accept pdf files");
        }
    }
}
```


⇒ Hascode returns integers representation of an object.

Q. Write a java program to demonstrate string comparison.

Q. Compare two methods returns an integer value which can be 0, +ve, -ve values.

~~if it is 0~~, It calculates the difference between the ~~asc~~ values of characters.

If it is 0 - they are equal, otherwise +ve or -ve.

Q. Write a java program to check whether given string is a palindrome or not?

⇒ Strings are immutable in nature.

When string objects are reinitialized the operating system creates a new memory location and the value gets updated in the new memory location and old memory location is garbage collected i.e; we cannot change the string once it is assigned, this is an overhead to the operating system to perform the above process. To overcome this drawback

java developers have introduced a new class called as StringBuffer class which is immutable in nature. The StringBuffer methods are synchronized and thread safe but slower in performance.

to improve the performance java developers have introduced new class called as `StringBuilder` (JDK-1.5 version) which gives greater performance. The methods of both the above classes are same, but the only difference is `StringBuilder` methods are not synchronized and they are not thread safe. `StringBuffer` methods are used in single threaded environment, whereas `StringBuilder` methods are used in multithreaded environment.

Q. Write a java program to demonstrate `StringBuilder` methods.

06-08-2026

Multithreading

The process of executing multiple tasks is called as multitasking. To achieve multitasking in java style we implement multithreading.

A thread is a lightweight process. A thread is a sequential flow of instructions that can run independently of each other. A thread is a sub process. It is a subpart of a program.

A thread does not exist on its own.

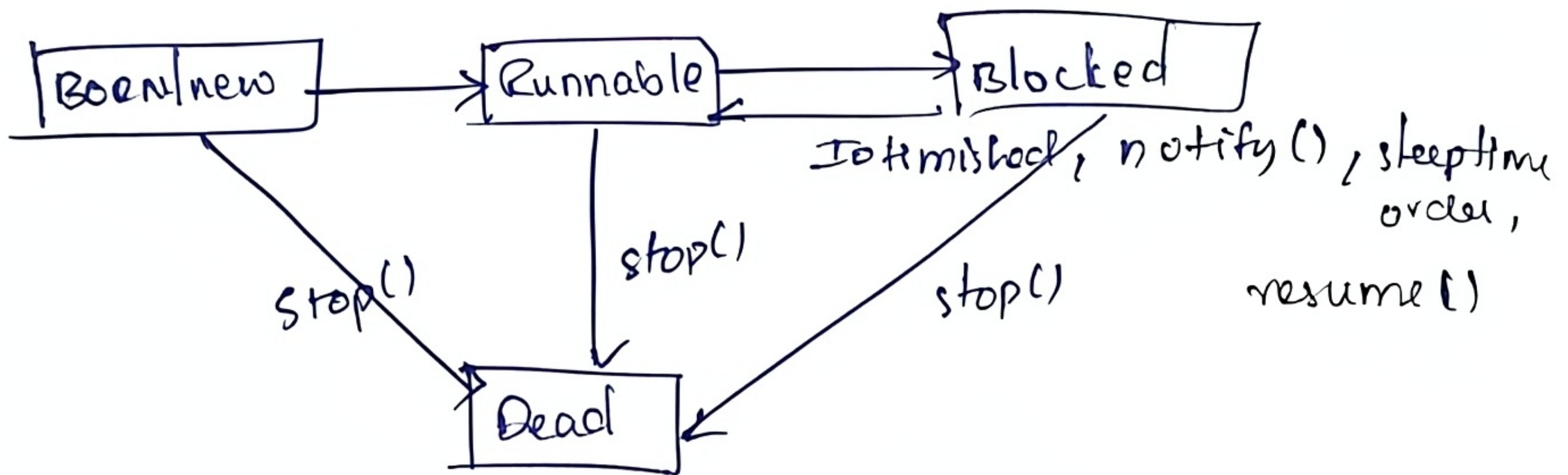
It runs along with the program.

Every $\Rightarrow$ Program divided into small blocks of code and each block is assigned as a thread. Every thread has its own execution path. All the threads share the memory space allocated to that process.

2) Life cycle of a thread

07-08-2026

$\Rightarrow$ Blocked; wait, sleep(), respond()



$\Rightarrow$ BORN state :-

When thread is created it is in born state. It is inactive and occupies some memories in the RAM.

`Thread t1 = new Thread();`

Runnable:-

To make the thread active call the start() which implicitly calls the run().

run() is considered as a heart of thread class.

The run() should contain block of statements that a thread has to be executed.

`t1.start()`

Blocked state:- We can make a active thread to inactive which goes to Block state,

Whenever IO is blocked, invoking `wait`, `sleep()`, `suspend()` methods will make thread to go to block state.

Once, IO is finished or sleep time over or calling `resume` and `notify` will get back to runnable state.

Dead state:-

Whenever a thread completes its execution it automatically dies and goes to dead state.

A dead thread cannot be restart again.

from any state if we call `stop()` it goes to dead state.

Q. How many ways a thread can be created?

two ways -

1. Extending Thread class

2. Implementing Runnable Interface.

⇒ Write a java program to create a thread using thread class.

NOTE:-

Whenever a class extends thread class directly it becomes a thread and it can access all the methods of a thread class. but when a class implements ^{interface} runnable it is not treated as thread.

Create an object of that class and pass the object as a parameter to the thread class constructor and with thread class object call the start method.

⇒ If a class is already extending another class it cannot extend thread class as java doesnot support multiple inheritance through classes. In this situation we implements RunnableInterface

11-08-26

Q Write a java program to create multiple threads the first thread has to print 1 to 10 and second thread has to print 10 to 1

Q. Write a java program to demonstrate thread class methods.

Daemon threads

Daemon threads are low priority thread which are service oriented threads which run at the background.

Example: Garbage collection

by default, All threads are non daemon threads. It can make daemon thread by calling a method `setDaemon(true)` and it should be called before the `start()` else `IllegalThreadStateException` will be thrown

⇒ Thread priorities

12-08-26

the thread scheduler allocates cpu utilization time based on some factors which can be waiting period of a thread, priority of a thread. In java, there are three priorities which should be in the range of 1-10. The following are the thread priority concepts:

`Thread.MAX_PRIORITY` — 10

`Thread.NORM_PRIORITY` — 5 (default)

`Thread.MIN_PRIORITY` — 1

By default, the threads which we create will have default priority has 5. ~~We~~ can change the priority by invoking `setPriority()`

Note:-

Setting the priority is a request to the operating system. the priority values must be between 1-10 else IllegalArgumentException will be thrown.

⇒ Inter thread Communication.

1. `yield()` → syntax: `this.yield();`

If a thread doesn't get a CPU utilization.

for a long period of time becomes a starving thread and ~~it~~ may die before execution.

the `yield()` is called on the current execution thread so that the thread scheduler will reschedule the CPU allocation time for these starving threads.

2. `join()`

`join` method will allow child threads to die first followed by parent thread (main method)

Join method throws Interrupted Exception which should be try catch block.

3. wait(), notify(), notifyAll()

These three methods belongs to Object class which are inherited by Thread class these methods are used with synchronization. The wait() is used to acquire a lock on the shared resource. ~~that~~ the notify(), notifyAll() are used to notify to the waiting threads to access the shared resource.

⇒ Synchronization

It allows threads to access the shared resources one after the other which will lead to accurate output and it will also avoid deadlock.

→ Synchronization can be achieved using two ways
1. declare a method as synchronized 2) use synchronized block.
Write a java program to demonstrate synchronization.

class Table

```
{ public void print(int n)
```

```
{ synchronized (this)
```

```
{ int i;
```

```
for (i=0; i<=10; i++)
```

```
{
```

```
System.out.println (n + "*" + i + "=" + (n*i));
```

```
}
```

```
}
```

```
}
```

```
}
```

class Five extends Thread

```
{ Table t;
```

```
public Five (Table t) {  
    this.t = t;
```

```
}  
public void run()
```

```
{  
    t.print(5);
```

```
}
```

```
}
```

class Ten extends Thread

```
{ Table t;
```

```
public Ten (Table t) {
```

```
    this.t = t;
```

```
}
```

```
public void run()
```

```
{ t.print(10);
```

```
}
```

```
public class Unsynch.
```

```
{ public static void main  
    (String[] args)
```

```
throws InterruptedException
```

```
{ Table b = new Table();
```

```
Five f = new Five(b);
```

```
Ten n = new Ten(b);
```

```
f.start();
```

```
n.start();
```

```
f.join();
```

```
n.join();
```

```
System.out.println
```

```
("main method after child  
thread");
```

```
}
```